

Estado	Finalizado
Comenzado	jueves, 11 de junio de 2026, 16:07
Completado	jueves, 11 de junio de 2026, 16:20
Duración	13 minutos 41 segundos
Puntos	13,00/13,00
Calificación	10,00 de 10,00 (100%)



Información

El programa utiliza 4 threads para calcular la suma total de los elementos de un vector A de gran tamaño, al tiempo que ejecuta una tarea auxiliar que actualiza un contador global (Tareas_Ejecutadas).

```
#include <stdio.h>
#include <omp.h>
#define N 100000
int main() {
    double A[N];
    double Suma_Total = 0.0;
    int Tareas_Ejecutadas = 0; // Variable compartida por defecto
    // Inicialización secuencial
    for (int i = 0; i < N; i++) {
        A[i] = i * 0.5;
    }
    // REGION PARALELA: Lanza un equipo de 4 threads
    #pragma omp parallel num_threads(4)
    {
        // Bloque 1: Tarea de inicio (solo un thread)
        #pragma omp single
        {
            printf("--- INICIO --- Thread %d preparó la ejecucion.\n", omp_get_thread_num());
        }
        // Bloque 2: Paralelismo de Datos (Work-Sharing)
        #pragma omp for reduction(+:Suma_Total) schedule(static,3)
        for (int i = 0; i < N; i++) {
            Suma_Total += A[i];

            // Simulación de actualización de estado que requiere protección
            if (i % 10000 == 0) {
                #pragma omp atomic
                Tareas_Ejecutadas++;
            }
        }
        // Bloque 3: Sincronización forzada
        #pragma omp barrier
        // Bloque 4: Tarea ejecutada solo por el thread maestro (ID 0)
        #pragma omp master
        {
            printf("--- FIN --- El thread %d (Maestro) recolectó los resultados.\n",
omp_get_thread_num());
        }
    } // Fin de la región paralela (JOIN)

    printf("Suma Total Calculada: %.2f\n", Suma_Total);
    printf("Contador final (Tareas Ejecutadas): %d\n", Tareas_Ejecutadas);
    return 0;
}
```



Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

¿Qué modelo de ejecución fundamental aplica la directiva `#pragma omp parallel` en el código, y qué sucede al finalizar el bloque de código justo antes de la línea `// Fin de la región paralela (JOIN)`

- ☒ a. Fork-Join. El thread maestro crea (fork) un equipo de threads para ejecutar el bloque, y al finalizar, todos los threads se sincronizan (join) para que la ejecución secuencial continúe. ✔
- ☐ b. SPMD (Single Program, Multiple Data). El programa falla porque SPMD solo se usa en MPI.
- ☐ c. Maestro-Eslavo. Solo el thread maestro (ID 0) ejecuta el código, el resto permanece inactivo.
- ☐ d. Pipeline de datos (DLP). Los threads continúan ejecutando el código secuencial sin esperar al maestro.

Respuesta correcta

Pregunta 2

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuál es el propósito de la cláusula `reduction(+:Suma_Total)` utilizada en el Bloque 2, y por qué se utiliza en lugar de una sincronización explícita como `critical`?

- ☐ a. Asegura que `Suma_Total` conserve el valor que tenía antes de entrar en la región paralela.
- ☐ b. Es necesaria porque `Suma_Total` fue declarada fuera de la región paralela.
- ☐ c. Garantiza que la variable `Suma_Total` sea compartida, previniendo el overhead del `schedule(dynamic)`.
- ☒ d. Crea una copia privada de la variable `Suma_Total` para cada thread y, al finalizar el bucle, combina todos los valores parciales (sumándolos) de forma segura en la variable original `Suma_Total`. Esto evita las condiciones de carrera sin la lentitud de un bloqueo manual (como `critical`). ✔

Respuesta correcta



Pregunta 3

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuál es la salida del programa si se elimina la cláusula `reduction(+:Suma_Total)`?

- ☐ a. No afecta el resultado.
- ☐ b. El resultado es:
Suma Total Calculada: 1504689587.00
Contador final (Tareas Ejecutadas): 10
- ☐ c. El resultado es:
Suma Total Calculada: 1503932947.50
Contador final (Tareas Ejecutadas): 10
- ☒ d. El valor de Contador final (Tareas Ejecutadas) es 10, pero el valor de Suma Total Calculada varía en cada ejecución porque ocurre una condición de carrera. ✓

Respuesta correcta

Pregunta 4

Correcta

Se puntúa 1,00 sobre 1,00

En el Bloque 2, la actualización del contador `Tareas_Ejecutadas` utiliza la directiva `#pragma omp atomic`. Si esta directiva se elimina, ¿qué ocurriría, y por qué es `atomic` más eficiente que usar `#pragma omp critical` en este caso?

- ☐ a. El `if` condicional causaría divergencia de flujo dentro del `warp`.
- ☐ b. Ocurriría un deadlock porque el `schedule(dynamic)` es bloqueante.
- ☐ c. Ocurriría un error de segmentación porque la variable `Tareas_Ejecutadas` no estaría inicializada.
- ☒ d. Ocurriría una Condición de Carrera (Race Condition), donde múltiples threads intentarían leer, modificar y escribir la variable compartida `Tareas_Ejecutadas` simultáneamente, llevando a resultados incorrectos. `Atomic` es más eficiente que `critical` porque está optimizada para operaciones simples y de actualización de memoria. ✓

Respuesta correcta



Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

En el Bloque 2, se utiliza la cláusula `schedule(static,3)` en el bucle `for`. ¿En qué escenario es preferible el `schedule` dinámico sobre el estático?

- ☐ a. Cuando el bucle tiene una sentencia `break` o `return`.
- ☐ b. Cuando el número de threads es mucho mayor que el número de iteraciones.
- ☒ c. Cuando las iteraciones del bucle tienen un tiempo de ejecución diferente y no predecible, permitiendo que los threads que terminan antes tomen nuevas porciones de trabajo, mejorando así el balance de carga. ✓
- ☐ d. Cuando todas las iteraciones del bucle tienen exactamente el mismo costo computacional, minimizando el overhead.

Respuesta correcta



Información

El objetivo es calcular la suma total de los elementos de un vector muy grande (A_global) dividido entre 4 procesos que se ejecutan en diferentes nodos de un clúster. Para lograr esto, el proceso raíz (rank 0) distribuye porciones iguales del vector a todos los demás procesos, que realizan su suma local y luego envían sus resultados parciales de vuelta al proceso raíz para la agregación final.

```
#include <mpi.h>
#include <stdio.h>

#define N_GLOBAL 1000 // Tamaño total del vector

int main(int argc, char** argv) {
    int rank, size;
    // 1. Inicializar el entorno MPI
    MPI_Init(&argc, &argv);

    // Obtener la identidad del proceso y el tamaño del comunicador
    MPI_Comm_rank(MPI_COMM_WORLD, &rank); // Rank (ID)
    MPI_Comm_size(MPI_COMM_WORLD, &size); // Size (NP)

    // Definir el tamaño local para cada proceso
    int N_LOCAL = N_GLOBAL / size;
    int A_global[N_GLOBAL];
    int A_local[N_LOCAL];

    // El proceso root (rank 0) inicializa los datos
    if (rank == 0) {
        for (int i = 0; i < N_GLOBAL; i++) {
            A_global[i] = 1; // Cada elemento es 1. Suma total esperada = 1000
        }
    }

    // 2. Distribución de Datos (del Root a Todos)
    MPI_Scatter(A_global, N_LOCAL, MPI_INT, A_local, N_LOCAL, MPI_INT, 0, MPI_COMM_WORLD); //

    // 3. Cómputo Local (cada proceso suma su porción)
    int suma_parcial = 0;
    for (int i = 0; i < N_LOCAL; i++) {
        suma_parcial += A_local[i];
    }

    // 4. Agregación de Resultados (Todos-a-Uno con operación)
    int suma_final_total = 0;
    MPI_Reduce(&suma_parcial, &suma_final_total, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD); //

    if (rank == 0) {
        printf("Resultado: Suma Total = %d\n", suma_final_total);
    }

    // 5. Finalizar el entorno MPI
    MPI_Finalize(); //
    return 0;
}
```



Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

¿Qué modelo de paralelismo fundamental utiliza MPI, y cuál es la información clave que MPI_Comm_rank y MPI_Comm_size proporcionan a cada proceso para que pueda colaborar?

- ☒ a. SPMD (Single Program, Multiple Data). Proporciona el Rank (ID) de cada proceso y el Size (NP), que es el número total de procesos en el comunicador. ✓
- ☐ b. TLP (Task Level Parallelism). Proporciona la etiqueta (tag) del mensaje y la función de broadcast.
- ☐ c. MIMD (Multiple Instruction, Multiple Data). Proporciona la dirección de memoria compartida y el número de threads.
- ☐ d. Fork-Join. Proporciona el tiempo de ejecución secuencial y el tiempo paralelo.

Respuesta correcta

Pregunta 7

Correcta

Se puntúa 1,00 sobre 1,00

En el Bloque 4, la función MPI_Reduce se utiliza para combinar los resultados parciales (suma_parcial). Si en su lugar se utiliza MPI_Allreduce, ¿cuál sería el impacto inmediato en el resultado del programa?

- ☒ a. El resultado final (suma_final_total) se calcularía, pero sería entregado a todos los procesos del comunicador, no solo al proceso root (rank 0). ✓
- ☐ b. La operación fallaría porque MPI_Allreduce no admite la operación MPI_SUM.
- ☐ c. Solo el proceso 0 obtendría el resultado, pero la operación sería más lenta.
- ☐ d. Se produciría un deadlock porque MPI_Allreduce es síncrona.

Respuesta correcta



Pregunta 8

Correcta

Se puntúa 1,00 sobre 1,00

En la comunicación punto a punto, si el Proceso 1 ejecuta un MPI_Send bloqueante al Proceso 2, y al mismo tiempo el Proceso 2 ejecuta un MPI_Send bloqueante al Proceso 1, sin que ninguno haya llamado a MPI_Recv, ¿qué error se producirá y por qué?

- ☐ a. Un error de sintaxis porque los mensajes no tienen la misma etiqueta (tag).
- ☐ b. Un error de balance de carga porque no se utilizaron funciones no bloqueantes.
- ☐ c. Un speed-up super-lineal debido al solapamiento de la comunicación.
- ☒ d. Un Deadlock (interbloqueo) porque las funciones MPI_Send bloqueantes esperan hasta que el mensaje haya sido recibido o almacenado en un búfer seguro. ✓

Respuesta correcta

Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

Si en el punto 3 se modifica `for (int i = 0; i < N_LOCAL; i++)` por `for (int i = 0; i < N_GLOBAL; i++)`, ¿qué sucede?

- ☐ a. Da error de ejecución.
- ☐ b. El programa se ejecuta correctamente.
- ☒ c. Entrega resultados erróneos porque los procesos están accediendo a zonas de memoria inválidas. ✓
- ☐ d. El programa no compila.

Respuesta correcta



Información

El programa implementa la operación para un vector de elementos, utilizando la arquitectura Host (CPU) para el control de flujo y el Device (GPU) para el cómputo paralelo.

```
#include <stdio.h>
#include <cuda_runtime.h>
#define N 1024

// =====
// Kernel CUDA: Se ejecuta en la GPU (Device), llamado desde la CPU (Host)
// =====

__global__ void vectorAdd(float *A, float *B, float *C, int N) {
    // Cálculo del índice global 'i' (1D)
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Cada thread procesa un elemento
    if (i < N)
        C[i] = A[i] + B[i];
}

int main() {
    size_t size = N * sizeof(float);

    // 1. Host (CPU) - Reserva e Inicialización
    float *h_A = (float*) malloc(size); // Memoria Host
    float *h_B = (float*) malloc(size);
    float *h_C = (float*) malloc(size);

    for (int i = 0; i < N; i++) {
        h_A[i] = i * 1.0f;
        h_B[i] = i * 2.0f;
    }

    // 2. Device (GPU) - Reserva de memoria
    float *d_A, *d_B, *d_C;
    cudaMalloc((void*)&d_A, size);
    cudaMalloc((void*)&d_B, size);
    cudaMalloc((void*)&d_C, size);

    // 3. Copia de datos: Host a Device
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // 4. Configuración y Llamada al Kernel
    int threadsPerBlock = 256;
    // blocksPerGrid = (1024 + 256 - 1) / 256 = 4 bloques
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    vectorAdd<<<blocksPerGrid, threadsPerBlock>>>>(d_A, d_B, d_C, N);

    // 5. Sincronización (Crucial)
    cudaDeviceSynchronize();
}
```



```
// 6. Copia de resultados: Device a Host
cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

printf("Resultado C = %.2f\n", h_C[N - 1]);

// 7. Liberación de memoria
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
free(h_A); free(h_B); free(h_C);
return 0;
}
```

Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

En el modelo de programación CUDA, ¿cuál es el rol del calificador `__global__` y qué elemento de la arquitectura representa el parámetro `blocksPerGrid` en la llamada al kernel?

- ☐ a. (b) `__global__` define una función que se ejecuta en la CPU (`__host__`). `blocksPerGrid` indica el tamaño del Warp.
- ☒ b. (c) `__global__` define el kernel, la función que se ejecuta en la GPU (Device) y es llamada desde la CPU (Host). `blocksPerGrid` define el número total de Bloques que componen la Malla (Grid), la ejecución total del kernel. ✓
- ☐ c. (a) `__global__` define una función auxiliar (`__device__`) llamada por la CPU. `blocksPerGrid` es el Work-item de OpenCL.
- ☐ d. (d) `__global__` se usa para variables en la memoria global. `blocksPerGrid` es el número de threads por bloque.

Respuesta correcta

Pregunta 11

Correcta

Se puntúa 1,00 sobre 1,00

¿Cuál es la secuencia correcta de operaciones que la CPU (Host) debe ejecutar para que la GPU (Device) pueda realizar la suma de vectores A y B?

- ☒ a. Reservar memoria en el Host (malloc) → Reservar memoria en el Device (cudaMalloc) → Copiar datos del Host al Device (cudaMemcpy HostToDevice). ✓
- ☐ b. Lanzar el kernel → cudaMemcpy (HostToDevice) → cudaMalloc.
- ☐ c. La GPU accede directamente a la memoria RAM de la CPU porque usan el mismo bus PCIe.
- ☐ d. malloc en el Host → Lanzar el kernel → cudaDeviceSynchronize.

Respuesta correcta



Pregunta 12

Correcta

Se puntúa 1,00 sobre 1,00

La instrucción `cudaDeviceSynchronize()` se coloca después de la llamada al kernel (`vectorAdd<<<...>>>`). ¿Qué sucede si se elimina esta llamada antes de `cudaMemcpy (DeviceToHost)`?

- ☐ a. Es crucial para liberar la memoria reservada en el Host.
- ☐ b. Se utiliza para evitar la divergencia de flujo dentro de los warps.
- ☒ c. La ejecución de los kernels en la GPU es asíncrona respecto a la CPU. `cudaDeviceSynchronize()` bloquea la ejecución del Host hasta que todas las tareas en la GPU hayan finalizado, garantizando que los resultados estén listos y evitando que la CPU lea datos no actualizados o incorrectos. ✓
- ☐ d. Es necesaria para calcular el índice global del thread (`blockIdx.x * blockDim.x + threadIdx.x`).

Respuesta correcta

Pregunta 13

Correcta

Se puntúa 1,00 sobre 1,00

En el kernel, la expresión `int i = blockIdx.x * blockDim.x + threadIdx.x`; se utiliza para:

- ☒ a. Calcular el índice global 1D (i) del elemento de datos que cada thread debe procesar. ✓
- ☐ b. Determinar la memoria compartida disponible para el bloque.
- ☐ c. Establecer el tamaño del Grid para la ejecución.
- ☐ d. Calcular el número de warps por Streaming Multiprocessor (SM).

Respuesta correcta

